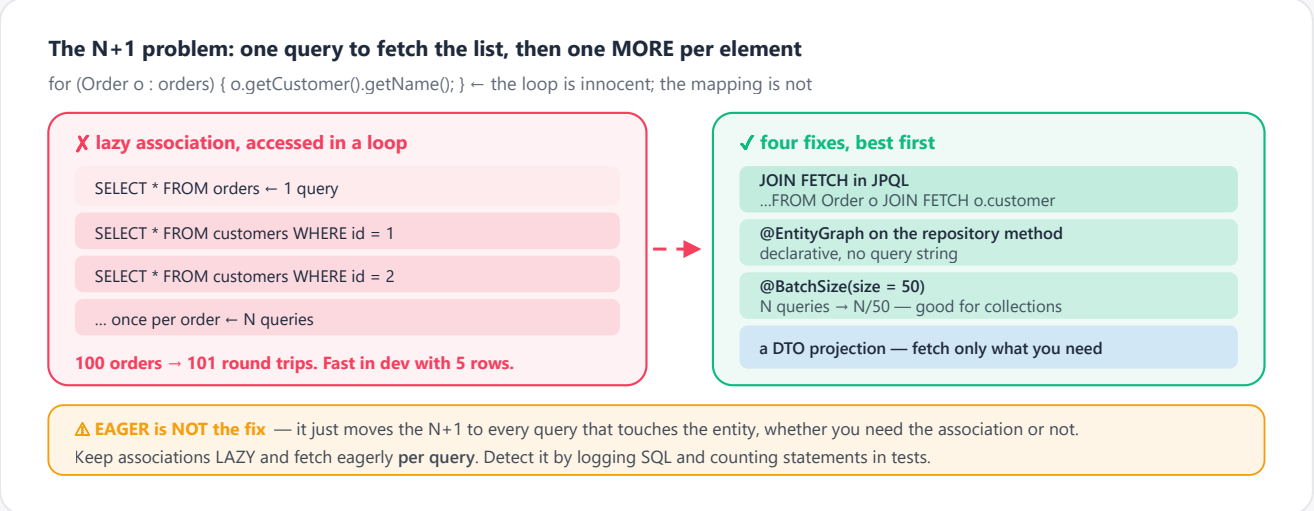
Lazy Loading and the N+1 Problem

Why it matters

N+1 is the most common performance bug in Java applications, and the most invisible: the code is clean, the tests pass, and it is fast in development with five rows of test data. In production, a page that should issue one query issues four hundred. Every JPA interview covers it, and being able to name four different fixes — and say why `EAGER` is not one of them — is a strong answer.

The concept, simply

Lazy loading means an association is not fetched until touched; Hibernate hands you a proxy and issues a query when you use it. That is the right default — but touching a lazy association inside a loop issues one query *per iteration*.



One query for the list, then N more. The loop looks innocent — the cost is entirely in how the association was mapped and fetched.

How it works

surprises almost everyone, and the eager to-one default is a common hidden cause of over-fetching.

The four fixes, in order of preference:

Fix	When
`JOIN FETCH` in JPQL	You control the query and know what you need
`@EntityGraph`	Declarative, on a derived repository method
`@BatchSize`	Collections — turns N queries into N/size
DTO projection	You only need a few columns — best of all

`EAGER` **is not a fix.** It moves the problem: the association is now fetched by *every* query touching the entity, whether or not that code path needs it. The correct approach is to keep everything `LAZY` and fetch eagerly per query.

Code examples

JAVA

Copy

```java
// ✗ N+1. One query for orders, then one per order for the customer.
List<Order> orders = repository.findByStatus(NEW);
for (Order o : orders) log.info(o.getCustomer().getName());

// ✓ 1. JOIN FETCH — one query, explicit
@Query("SELECT o FROM Order o JOIN FETCH o.customer WHERE o.status = :stat
List<Order> findByStatusWithCustomer(@Param("status") OrderStatus status);

// ✓ 2. @EntityGraph — same effect, declarative, works with derived metho
@EntityGraph(attributePaths = {"customer", "lines"})
List<Order> findByStatus(OrderStatus status);

// ✓ 3. @BatchSize on the association or entity — N queries become N/50
@OneToMany(mappedBy = "order")
```

```java
// ✓ 4. A projection — fetch two columns instead of whole object graphs
@Query("SELECT new com.app.OrderSummary(o.reference, c.name) " +
       "FROM Order o JOIN o.customer c WHERE o.status = :status")
List<OrderSummary> findSummaries(@Param("status") OrderStatus status);
```

The trap in fetch-joining more than one collection:

JAVA Copy

```java
// ✗ MultipleBagFetchException — you cannot JOIN FETCH two List collection
@Query("SELECT o FROM Order o JOIN FETCH o.lines JOIN FETCH o.payments")

// ✓ Option A: make them Sets rather than Lists.
// ✓ Option B: fetch one and use @BatchSize for the other.
// ✓ Option C: two queries — the persistence context stitches them togethe
//   because the identity guarantee means the second query populates the
//   SAME objects the first returned.

// ⚠ Also: JOIN FETCH plus pagination silently loads EVERYTHING into memo
//   and paginates there. Hibernate logs
//   "firstResult/maxResults specified with collection fetch; applying in
//   — treat that warning as an error. Use @BatchSize or two queries inste
```

Catching it before production, which is the answer interviewers want:

PROPERTIES Copy

```properties
# Development only — never in production, it is enormous and slow.
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
logging.level.org.hibernate.SQL=DEBUG

# Better: fail the build. Datasource-proxy or the Hypersistence utilities
# can assert a query COUNT in a test:
```

Under the hood

Hibernate implements lazy loading with **proxies** — a generated subclass for a to-one association, a lazy wrapper for a collection. This is why entity classes and their getters must not be `final`, and why `getClass()` in an entity's `equals` fails against a proxy.

Why the problem is invisible in tests: with ten rows, 11 queries against an in-memory H2 take microseconds. With ten thousand rows against a database across a network, 10,001 round trips at even 1 ms each is over ten seconds. The bug does not scale linearly into visibility — it arrives all at once with real data volume. That is the argument for asserting query counts rather than relying on noticing.

The wider lesson is worth stating in an interview: an ORM hides the database, and hiding it is exactly why these problems occur. The habit that prevents nearly all of them is simply to look at the generated SQL during development — most JPA performance bugs are visible in the log the first time the code runs.

Common mistakes

- **Switching to `EAGER` to fix N+1.** Moves the cost to every query instead.
- **Not knowing `@ManyToOne` is eager by default.** Silent over-fetching everywhere.
- **Fetch-joining two collections.** `MultipleBagFetchException`.
- **Combining `JOIN FETCH` with pagination.** Paginates in memory after loading everything.
- **Never looking at the SQL.** Nearly every JPA performance bug is visible in the log.
- **Returning entities from controllers.** Serialisation touches lazy associations and triggers N+1 in the web layer.
- **Relying on tests with tiny datasets.** Assert query counts instead.

Interview lens

A: One query loads a list of N entities, then touching a lazy association on each issues one more query per element — N+1 in total. The loop looks completely innocent; the cost comes from the mapping and the fetch strategy. It is invisible in development because ten rows against an in-memory database is fast, and catastrophic in production with real volume and network latency.

Q: "How do you fix it?"

A: Four ways. `JOIN FETCH` in a JPQL query when I control the query. `@EntityGraph` for the same effect declaratively on a derived method. `@BatchSize`, which turns N queries into N divided by the batch size — good for collections. Or a DTO projection that selects only the columns needed, which is usually best because it avoids loading entities at all.

Q: "Why not just make it EAGER?"

A: Because eager fetching cannot be turned off per query. The association is then loaded by every query that touches the entity, including the many code paths that never use it — so you have replaced N+1 on one screen with permanent over-fetching everywhere. The right model is LAZY by default, fetched eagerly per query where needed.

Q: "What are the default fetch types?"

A: `@ManyToOne` and `@OneToOne` are EAGER; `@OneToMany` and `@ManyToMany` are LAZY. The to-one default catches people out constantly — it means every load of an entity silently drags its parents along unless you set `fetch = FetchType.LAZY` explicitly.

Q: "How would you detect it?"

A: Enable SQL logging in development and actually read it — most of these bugs are obvious the first time the code runs. Better, assert query counts in integration

surfaces it quickly.

Q: "What happens if you fetch-join a collection with pagination?"
A: Hibernate cannot apply the limit in SQL without losing rows, so it loads the entire result set into memory and paginates there — and logs a warning saying exactly that. On a large table it is an `OutOfMemoryError` waiting to happen. Use `@BatchSize` or a two-query approach, where the second query populates the same objects thanks to the persistence context's identity guarantee.

Cheat-card

- N+1 = 1 query for the list + **1 per element** for a lazy association.
- Defaults: `@ManyToOne` / `@OneToOne` **EAGER**, `@OneToMany` / `@ManyToMany` LAZY.
- Fixes: `JOIN FETCH` · `@EntityGraph` · `@BatchSize` · **DTO projection**.
- `EAGER` **is not a fix** — it over-fetches on every query instead.
- Two collection fetch-joins → `MultipleBagFetchException` (use Sets, or batch one).
- `JOIN FETCH` **+ pagination = in-memory pagination.** Treat the warning as an error.
- Detect with **SQL logging in dev + query-count assertions in tests**.
- Lazy loading uses **proxies** → entities and getters must not be `final`.

QUICK CHECK — no marks, just for you.

1. What are JPA's DEFAULT fetch types?

- ○ @ManyToOne and @OneToOne are EAGER; @OneToMany and @ManyToMany are LAZY
- ○ Everything is EAGER by default
- ○ @ManyToOne is LAZY; collections are EAGER

Check answer

2. Why is switching an association to EAGER not a fix for N+1?

- ◯ Because EAGER is slower per query
- ◯ Because eager fetching cannot be disabled per query, so the association is now loaded by EVERY query touching the entity — you have replaced N+1 on one screen with permanent over-fetching everywhere
- ◯ Because EAGER causes LazyInitializationException
- ◯ Because EAGER only works for collections

Check answer

3. What happens if you combine JOIN FETCH of a collection with pagination?

- ◯ It throws MultipleBagFetchException
- ◯ It silently ignores the pagination
- ◯ Hibernate loads the ENTIRE result set into memory and paginates there, logging a warning about applying firstResult/maxResults in memory
- ◯ Hibernate applies the limit in SQL as usual

Check answer

LEARN MORE

ARTICLES & DOCS

- JPA query methods and fetching — Spring Data JPA
  docs.spring.io
- Hibernate ORM user guide — fetching
  hibernate.org

VIDEOS



- Interview Q&A | Hibernate N+1 Problem & Fix - The Bug You Didn't Know You Had @Javatechie
  Java Techie



- JOIN FETCH with Hibernate - The end of n+1 select issues
  Thorben Janssen

Hand-picked references for this lesson.